

Industrial problem solving task

C04 AT3

1. Project Structure

recommendation_system/

|

└─ main.py

└─ data_collection.py

└─ analysis.py

└─ recommendation.py

└─ storage.py

└─ report.py

|

└─ data/

└─ user_activity.csv

└─ products.csv

└─ recommendations.json

2. Sample products.csv

product_id,product_name,category

P101,Smartphone,Electronics

P102,Laptop,Electronics

P103,Headphones,Electronics

P104,T-Shirt,Clothing

P105,Jeans,Clothing

P106,Washing Machine,Home Appliances

P107,Refrigerator,Home Appliances

P108,Smart Watch,Electronics

3. Sample user_activity.csv

user_id,product_id,action

U101,P101,view

U101,P102,view

U101,P101,purchase

U101,P103,view

U102,P104,view

U102,P105,purchase

U102,P104,purchase

U103,P101,view

U103,P108,purchase

4. Data Collection Module – data_collection.py

This module reads user browsing and purchase history.

```
import csv
```

```
def load_user_activity(filename):
```

```
    activities = []
```

```
    try:
```

```
with open(filename, "r", newline="") as file:
    reader = csv.DictReader(file)

    for row in reader:
        if row["user_id"] and row["product_id"] and row["action"]:
            activities.append(row)

except FileNotFoundError:
    print("Error: User activity file not found.")

except Exception as e:
    print("Error:", e)

return activities


def load_products(filename):
    products = []

    try:
```

```
with open(filename, "r", newline="") as file:
```

```
    reader = csv.DictReader(file)
```

```
    for row in reader:
```

```
        products.append(row)
```

```
except FileNotFoundError:
```

```
    print("Error: Product file not found.")
```

```
return products
```

5. Analysis Module – analysis.py

This module analyzes the user's behavior and finds frequently viewed or purchased products.

```
from collections import Counter
```

```
def analyze_user_preferences(activities, user_id):
```

```
    product_count = Counter()
```

```
    for activity in activities:
```

```
        if activity["user_id"] == user_id:
```

```
            product_id = activity["product_id"]
```

```
            action = activity["action"]
```

```
            # Purchase gets higher importance
```

```
            if action == "purchase":
```

```
                product_count[product_id] += 2
```

```
            else:
```

```
                product_count[product_id] += 1
```

```
    return product_count
```

```
def get_popular_products(activities):
```

```
product_count = Counter()
```

```
for activity in activities:
```

```
    product_count[activity["product_id"]] += 1
```

```
return product_count.most_common()
```

How it works

For example, if a user:

Views Smartphone → +1

Views Smartphone again → +1

Purchases Smartphone → +2

Then the preference score becomes:

Smartphone = 4

So the system understands that the user is highly interested in that product.

6. Recommendation Engine – recommendation.py

This module recommends similar products based on categories and user behavior.

```
def generate_recommendations(preferences, products,
                             limit=5):
```

```
    recommendations = []
```

```
    # Get products preferred by user
```

```
    preferred_products = list(preferences.keys())
```

```
    # Find categories of preferred products
```

```
    preferred_categories = set()
```

```
for product in products:
    if product["product_id"] in preferred_products:
        preferred_categories.add(product["category"])

# Recommend products from same categories
for product in products:

    if (product["category"] in preferred_categories and
        product["product_id"] not in preferred_products):

        recommendations.append({
            "product_id": product["product_id"],
            "product_name": product["product_name"],
            "category": product["category"]
        })

    if len(recommendations) >= limit:
        break

return recommendations
```

Example

If a user frequently views:

Smartphone

Laptop

Headphones

The preferred category is:

Electronics

The system may recommend:

Smart Watch

Other Electronics Products

7. Data Storage Module – storage.py

Recommendations are stored in a JSON file.

```
import json
```

```
def save_recommendations(user_id, recommendations):
```

```
    data = {}
```

```
    try:
```

```
        with open("data/recommendations.json", "r") as file:
```

```
            data = json.load(file)
```

```
    except (FileNotFoundError, json.JSONDecodeError):
```

```
        data = {}
```

```
    data[user_id] = recommendations
```

```
    try:
```

```
        with open("data/recommendations.json", "w") as file:
```

```
json.dump(data, file, indent=4)
```

```
print("Recommendations saved successfully.")
```

```
except Exception as e:
```

```
    print("Error saving recommendations:", e)
```

Example JSON output:

```
{
  "U101": [
    {
      "product_id": "P108",
      "product_name": "Smart Watch",
      "category": "Electronics"
    }
  ]
}
```

8. Report Generation Module – report.py

This generates a report of popular products.

```
def generate_report(popular_products, products):

    product_names = {}

    for product in products:
        product_names[product["product_id"]] = product["product_name"]

    print("\n----- POPULAR PRODUCTS REPORT -----")

    for product_id, count in popular_products:

        name = product_names.get(product_id, "Unknown Product")

        print(f"{name}: {count} interactions")
```

9. Main Program – main.py

This connects all the modules.

```
from data_collection import load_user_activity,  
load_products
```

```
from analysis import analyze_user_preferences,  
get_popular_products
```

```
from recommendation import generate_recommendations
```

```
from storage import save_recommendations
```

```
from report import generate_report
```

```
def main():
```

```
    # Load data
```

```
    activities = load_user_activity("data/user_activity.csv")
```

```
products = load_products("data/products.csv")
```

```
if not activities or not products:
```

```
    print ("Required data is missing.")
```

```
    return
```

```
# User input
```

```
user_id = input ("Enter User ID: ")
```

```
# Analyse preferences
```

```
preferences = analyse_user_preferences(activities, user_id)
```

```
if not preferences:
```

```
    print("No activity found for this user.")
```

```
    return
```

```
print("\nUser Preferences:")
```

```
for product, score in preferences.items():
```

```
    print(product, "Score:", score)
```

```
# Generate recommendations

recommendations = generate_recommendations(
    preferences,
    products
)

print("\n----- RECOMMENDED PRODUCTS -----")

if recommendations:

    for product in recommendations:
        print(
            product["product_name"],
            "-",
            product["category"]
        )

    else:
        print("No recommendations available.")

# Save recommendations
```

```
save_recommendations(user_id, recommendations)
```

```
# Generate popular products report
```

```
popular_products = get_popular_products(activities)
```

```
generate_report(popular_products, products)
```

```
if __name__ == "__main__":
```

```
    main()
```

```
---
```

10. Expected Output

Enter User ID: U101

User Preferences:

P101 Score: 3

P102 Score: 1

P103 Score: 1

----- RECOMMENDED PRODUCTS -----

Smart Watch - Electronics

Recommendations saved successfully.

----- POPULAR PRODUCTS REPORT -----

Smartphone: 3 interactions

T-Shirt: 2 interactions

Laptop: 1 interactions

Headphones: 1 interactions

How the Project Satisfies the Requirements

Requirement	Implementation
-------------	----------------

a) Data Collection	User activity stored in CSV files
--------------------	-----------------------------------

- b) Analysis Module Counts views and purchases using Counter
- c) Recommendation Engine Recommends products from preferred categories
- d) Data Storage Saves recommendations in JSON
- e) Report Generation Displays popular products and interactions
- f) Error Handling Handles missing files and invalid JSON
- g) Scalability Modular design and efficient Counter usage
- h) Extensibility Can later add collaborative filtering or machine learning

Future Enhancements

Later, this basic system can be extended with:

1. Collaborative Filtering – Recommend products liked by similar users.
2. Content-Based Filtering – Recommend products with similar features.

3. Machine Learning – Predict products a customer is likely to purchase.

4. Database Integration – Replace CSV files with MySQL or MongoDB.

5. Real-Time Recommendations – Update suggestions immediately when users browse or purchase products.